

Технологии параллельного программирования

OpenMP

OpenMP

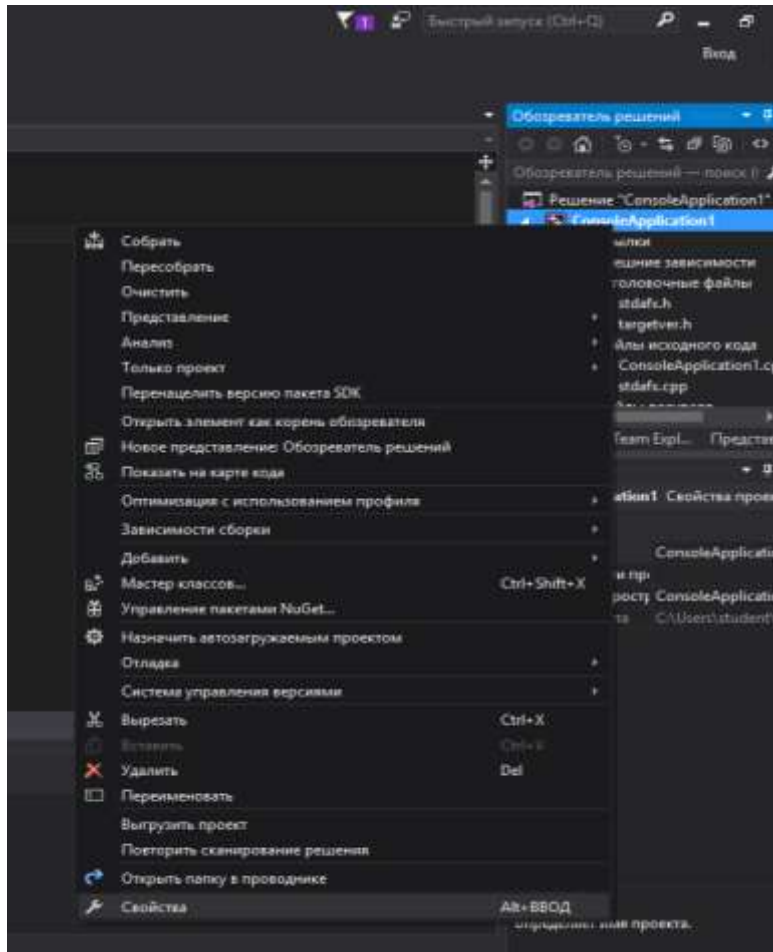
- OpenMP (Open Multi-Processing) — открытый стандарт для распараллеливания программ на языках Си, Си++ и Фортран. Дает описание совокупности директив компилятора, библиотечных процедур и переменных окружения, которые предназначены для программирования многопоточных приложений на многопроцессорных системах с общей памятью.
- Первая версия появилась в 1997 году, предназначалась для языка Fortran.
- Для C/C++ версия разработана в 1998 году.
- В 2008 году вышла версия OpenMP 3.0.

- OpenMP реализует параллельные вычисления с помощью многопоточности, в которой «главный» (master) поток создает набор подчиненных (slave) потоков и задача распределяется между ними.
- Предполагается, что потоки выполняются параллельно на вычислительной машине с несколькими процессорами (количество процессоров не обязательно должно быть больше или равно количеству потоков).
- Задачи, выполняемые потоками параллельно, также как и данные, требуемые для выполнения этих задач, описываются с помощью специальных директив препроцессора соответствующего языка — **прагм**.

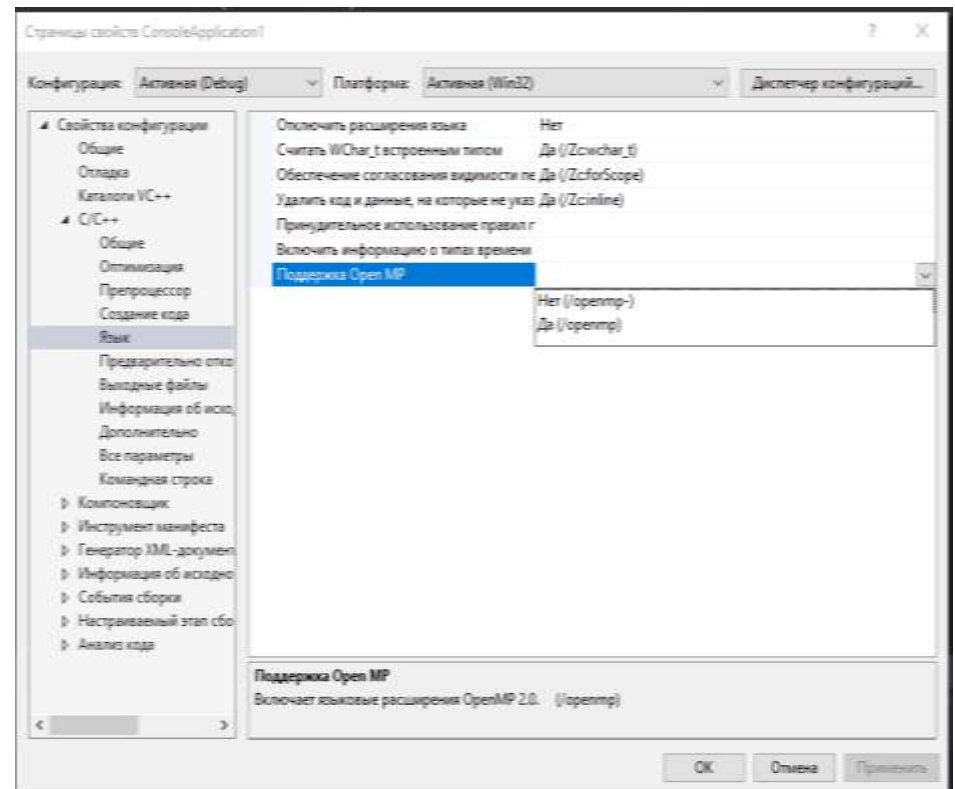
Существующие реализации

- Компиляторы Sun Studio поддерживают официальную спецификацию (OpenMP 2.5) с улучшенной производительностью под ОС Solaris.
- Visual C++ 2005 и 2008 поддерживает OpenMP 2.0 в редакциях Professional и Team System, 2010 - в редакциях Professional, Premium и Ultimate, 2012 и выше - во всех редакциях.
- GCC 4.2 поддерживает OpenMP, а некоторые дистрибутивы (такие как Fedora Core 5 gcc) включили поддержку в свои версии GCC 4.1.
- Intel C++ Compiler поддерживает версию OpenMP 3.0 а также Intel Cluster OpenMP для программирования в системах с распределённой памятью.
- IBM XL compiler.
- PGI (Portland group).
- И др.

Включение режима компиляции с поддержкой OpenMP в MS Visual Studio 2015



Шаг 1



Шаг 2

Функции

- Функции OpenMP носят скорее вспомогательный характер, так как реализация параллельности осуществляется за счет использования директив.
- Функции можно разделить на три категории: функции исполняющей среды, функции блокировки/синхронизации и функции работы с таймерами.
- Все эти функции имеют имена, начинающиеся с `omp_`, и определены в заголовочном файле **`omp.h`**.

Директивы

- Конструкция `#pragma` в языке Си/Си++ используется для задания дополнительных указаний компилятору. С помощью этих конструкций можно указать как осуществлять выравнивание данных в структурах, запретить выдавать определенные предупреждения и так далее.
- Форма записи:
`#pragma` директивы
- Использование специальной ключевой директивы «omp» указывает на то, что команды относятся к OpenMP. Таким образом директивы `#pragma` для работы с OpenMP имеют следующий формат:
`#pragma omp <директива> [раздел [ [,] раздел]...]`
- Как и любые другие директивы `pragma`, они игнорируются теми компиляторами, которые не поддерживают данную технологию. При этом программа компилируется без ошибок как последовательная.

Директива parallel

#pragma omp parallel [другие директивы]

структурированный блок

- Директива parallel указывает, что структурный блок кода должен быть выполнен параллельно в несколько потоков.
- Каждый из созданных потоков выполнит одинаковый код содержащийся в блоке, но не одинаковый набор команд.
- В разных потоках могут выполняться различные ветви или обрабатываться различные данные, что зависит от таких операторов как if-else или использования директив распределения работы.

Пример

Последовательный код

```
void main() {  
    double x[1000];  
    for(i=0; i<1000; i++){  
        calc_smoth(&x[i]);  
    }  
}
```

Параллельный код

```
void main() {  
    double x[1000];  
    #pragma omp parallel for ...  
    for(i=0; i<1000; i++){  
        calc_smoth(&x[i]);  
    }  
}
```

Параллельные регионы

- Параллельные регионы являются основным понятием в OpenMP.
- Именно там, где задан этот регион программа выполняется параллельно. Как только компилятор встречает прагму **omp parallel**, он вставляет инструкции для создания параллельных потоков.
- Количество порождаемых потоков для параллельных областей контролируется через переменную окружения **OMP_NUM_THREADS**, а также может задаваться через вызов функции внутри программы.

- Каждый порожденный поток исполняет блок код в структурном блоке. **По умолчанию синхронизация между потоками отсутствует** и поэтому последовательность выполнения конкретного оператора различными потоками не определена.
- После выполнения параллельного участка кода все потоки, кроме основного завершаются, и только основной поток продолжает исполняться, но уже один.
- Каждый поток имеет свой уникальный номер, который изменяется от 0 (для основного потока) до количества потоков – 1. Идентификатор потока может быть определен с помощью функции **omp_get_thread_num()**.

Пример

- Получение номера потока

```
#pragma omp parallel
{
    myid = omp_get_thread_num();
    if(myid == 0)
        do_something();
    else
        do_something_else(myid);
}
```

Модель исполнения

- Существует две модели исполнения:
динамическая, когда количество используемых потоков в программе может варьироваться от одной области параллельного выполнения к другой, и **статическая**, когда количество потоков фиксировано.
- Модель исполнения контролируется или через переменную окружения OPM_DYNAMIC или с помощью вызова функции `omp_set_dynamic()`.

Конструкции OpenMP для распределения работ

- параллельный цикл `for`;
- параллельные секции (`sections`);
- конструкция `single`.

Директива for

```
#pragma omp parallel
{
    #pragma omp for
    for (ptrdiff_t i = 0; i < n; i++)
        dst[i] = sqrt(src[i]);
}
```

Параллельный цикл

- По умолчанию барьером для потоков является **конец цикла**.
- Все потоки достигнув конца цикла дожидаются тех, кто еще не завершился, после чего основная нить продолжает выполняться дальше.
- Используя условие **nowait** для цикла, можно разрешить основной нити не дожидаться завершения дочерних нитей.

Параллельные секции

- Каждая секция выполняется в отдельном потоке, что позволяет производить декомпозицию по коду.
- Точкой синхронизации является конец блока sections.
- В случае, когда необходимо чтобы основной поток не ждал завершения остальных потоков следует использовать условие nowait.

Пример

```
#pragma omp sections \  
    [условие [,условие...]]  
{  
    #pragma omp section  
        структурный блок  
    [#pragma omp section  
        структурный блок  
    ...]  
}
```

где условие – это одно из:

```
private(var1, var2, ...)  
firstprivate(var1, var2, ...)  
lastprivate(var1, var2, ...)  
reduction(оператор: var1, var2, ....)  
nowait
```

Конструкция single

- Если в параллельной секции требуется выполнить какое-либо действие и при этом это действие должно быть выполнено только одним потоком (например, подсчет промежуточного результата), то для этого подходит конструкция single.

Пример

```
#pragma omp single [условие [, условие ...]]  
    структурный блок
```

где УСЛОВИЕ – это одно из:

```
private(var1, var2, ...)  
firstprivate(var1, var2, ...)  
nowait
```

Директивы `private` и `shared`

- Относительно параллельных регионов данные могут быть общими (`shared`) или частными (`private`).
- **Частные** данные принадлежат потоку и могут быть модифицированы только им.
- **Общие** данные доступны всем потокам.
- Если переменная объявлена вне параллельного региона, то по умолчанию она считается общей, а если внутри то частной.
- Директиву `shared` обычно не используют, так как и без нее все переменные объявленные вне параллельного региона будут общими. Ее можно использовать для повышения наглядности кода.

Средства синхронизации в OpenMP

- `critical` – критическая секция
- `atomic` – атомарность операции
- `barrier` – точка синхронизации
- `master` – блок, который будет выполнен только основным потоком. Все остальные потоки пропустят этот блок. В конце блока неявной синхронизации нет.
- `ordered` – выполнять блок в заданной последовательности
- `flush` – немедленный сброс значений разделяемых переменных в память.

Справочная информация по OpenMP

- <http://parallel.ru/sites/default/files/ftp/openmp/cspec.pdf>
- <http://ccfit.nsu.ru/arom/data/openmp.pdf>
- <http://openmp.org>